

Instruções

1. Esta avaliação deve ser feita em dupla ou em trio.
 2. Data de entrega: **09/06/2026** até as 19:00. Não pode entregar em atraso.
 3. O objetivo deste trabalho é evoluir o sistema de gerência de memória do projeto Microkernel, substituindo o alocador linear atualmente implementado por um algoritmo de alocação dinâmica baseado em Free List Allocator com política First Fit. Ao concluir este trabalho, o aluno deverá ser capaz de:
 - a. compreender gerenciamento dinâmico de memória;
 - b. implementar allocators baseados em lista livre;
 - c. compreender fragmentação;
 - d. implementar reutilização de memória;
 - e. integrar gerenciamento de memória a um microkernel;
 - f. comparar diferentes políticas de alocação.
 4. A implementação deverá ser desenvolvida utilizando a linguagem C devido a natureza do código fonte de referência.
 5. O sistema deve ser entregue funcionando corretamente. Sistemas não compilando e executando não serão aceitos. É de responsabilidade do(s) aluno(s) apresentar a execução funcionando corretamente.
 6. Deve ser disponibilizado os códigos da implementação em repositório do(s) aluno(s) (github, bitbucket, etc...), deve ser fornecido o link e o repositório deve ser público.
 7. O relatório, em formato PDF (não é aceito outro formato), deve seguir o formato de artigo científico ou seguindo as normas da ABNT e as orientações para produção de trabalhos acadêmicos da Univali contendo:
 - Identificação do autor e do trabalho.
 - Enunciado dos projetos.
 - Explicação e contexto da aplicação para compreensão do problema tratado pela solução.
 - Resultados obtidos com as simulações.
 - Códigos importantes da implementação.
 - Resultados obtidos com a implementação (tabelas, gráficos e etc).
 - Análise e discussão sobre os resultados finais.
 8. Deve ser disponibilizado os códigos da implementação (todos os arquivos novos e modificados) juntamente com o relatório (salvo o caso da disponibilidade em repositório aberto do aluno, que deve ser fornecido o link). O repositório deve estar aberto do momento da entrega em diante, sendo que o professor não responsabiliza caso o projeto não esteja disponível para consulta no momento da correção, sendo do(s) aluno(s) essa responsabilidade de manter disponível. Cópias entre alunos implicará em nota zero para todos os envolvidos.
 9. O trabalho deverá ser apresentado em data definida pelo professor. É de responsabilidade do(s) aluno(s) explicar os conceitos, comandos, bibliotecas usadas. É de responsabilidade do(s) aluno(s) fazer a solução funcionar e ela deverá ser baixada do local de entrega no momento da apresentação. Trabalhos não apresentados terão como nota máxima 5,0, além dos descontos aplicados no restante da avaliação da implementação. Todos os alunos poderão apresentar ou o professor poderá escolher um representante para apresentar o trabalho em nome do grupo.
-

Projeto

A proposta busca aproximar o microkernel de mecanismos utilizados em sistemas operacionais reais, permitindo ao aluno compreender conceitos fundamentais de gerenciamento dinâmico de memória em kernels. Atualmente, o microkernel utiliza um allocator linear (Bump Allocator), no qual a memória é alocada apenas avançando um ponteiro dentro do heap. Embora simples, esse modelo apresenta limitações importantes:

- não permite reutilização de memória;
- não possui mecanismo de desalocação;
- não implementa `kfree()`;
- gera exaustão progressiva do heap;
- não trata fragmentação;
- não permite gerenciamento dinâmico eficiente de tasks.

Neste trabalho, o aluno deverá implementar um allocator mais sofisticado baseado em blocos livres encadeados.

Repositório de Referência

O projeto-base utilizado neste trabalho está disponível em:

<https://github.com/VielF/BasicMicrokernel>

O repositório contém:

- código-fonte completo do microkernel;
- sistema de build;
- linker script;
- gerenciamento atual de memória;
- scheduler;
- troca de contexto;
- documentação completa do projeto.

Leitura Obrigatória

Antes de iniciar o desenvolvimento, o aluno deverá estudar cuidadosamente:

- o código-fonte do módulo atual de memória;
- os capítulos da documentação referentes:
 - ao heap;
 - à criação de tasks;
 - ao allocator atual;
 - ao scheduler;
 - à organização da memória no linker script.

Além disso, o aluno deverá utilizar como base o documento de descrição fornecido pelo professor contendo:

- explicação teórica do Free List Allocator;
- estrutura esperada dos blocos;
- fluxo de alocação e desalocação;
- funcionamento da política First Fit;
- conceitos de fragmentação e coalescência;
- esqueleto base do código.

Descrição da Proposta

O novo allocator deverá:

- dividir o heap em blocos;
- armazenar metadados em cabeçalhos;
- manter uma lista encadeada de blocos;
- reutilizar memória liberada;
- implementar divisão de blocos;
- implementar coalescência de blocos adjacentes;
- fornecer estatísticas de uso do heap.

Estrutura Esperada do Bloco

Cada bloco deverá possuir um cabeçalho semelhante ao seguinte:

```
typedef struct block
{
    uint64_t size;
    int free;
    struct block *next;
} block_t;
```

Funcionalidades Obrigatórias

O aluno deverá implementar obrigatoriamente:

- Inicialização do Heap
- Implementar:
- void memory_init(void);

Responsabilidades:

- inicializar a lista livre;
- configurar o primeiro bloco;
- preparar o heap para alocação dinâmica.

Alocação Dinâmica

Implementar:

```
void *kmalloc(uint64_t size);
```

O algoritmo deverá:

- utilizar política First Fit;
- percorrer a lista de blocos;
- localizar blocos livres;
- dividir blocos quando necessário;
- retornar ponteiro para a área útil.

Desalocação

Implementar:

```
void kfree(void *ptr);
```

A função deverá:

- recuperar o cabeçalho do bloco;
- marcar o bloco como livre;
- implementar coalescência.

Estatísticas do Heap

Implementar:

```
uint64_t memory_used(void);
uint64_t memory_free(void);
uint64_t memory_total(void);
```

Integração com Tasks

O sistema de criação de tasks deverá ser modificado para utilizar o novo allocator. As stacks das tasks deverão ser alocadas dinamicamente por meio de:

`kmalloc()`

Além disso, o sistema deverá permitir a liberação da memória das stacks utilizando:

`kfree()`

Requisitos Técnicos

A implementação deverá:

- ser escrita em C;
- funcionar no ambiente RISC-V do projeto;
- ser compatível com o QEMU;
- manter compatibilidade com o restante do microkernel;
- não utilizar bibliotecas externas;
- não utilizar `malloc()` da `libc`.

Requisitos de Funcionamento

Após a implementação:

- o kernel deverá continuar executando normalmente;
- as tasks deverão funcionar corretamente;
- o heap deverá reutilizar memória liberada;
- o sistema deverá permitir múltiplas alocações e liberações sem reinicialização.

Funcionalidades Extras (Opcional)

Os seguintes recursos podem ser implementados como extensão:

- alinhamento automático de memória;
- cálculo de fragmentação;
- destruição dinâmica de tasks;
- benchmark de alocação;
- implementação de Best Fit ou Worst Fit;
- visualização do mapa do heap.

Testes Esperados

O aluno deverá criar cenários demonstrando:

1. múltiplas alocações;
2. liberação de memória;
3. reutilização de blocos;
4. divisão de blocos;
5. coalescência;
6. estatísticas do heap.

Exemplo de Saída Esperada

Heap total: 65536 bytes

Heap usado: 4096 bytes

Heap livre: 61440 bytes

Task 1 criada

Task 2 criada

Task 1 removida

Heap usado: 2048 bytes

Heap livre: 63488 bytes

Entregáveis

O aluno deverá entregar:

- todos os arquivos modificados;
- novos arquivos criados;
- relatório técnico contendo:
 - explicação da implementação;
 - decisões de projeto;
 - dificuldades encontradas;
 - testes realizados;
 - capturas da execução no QEMU.

Obs: O documento de descrição fornecido pelo professor deve ser utilizado como principal referência de implementação. O objetivo do trabalho não é apenas reproduzir código, mas compreender profundamente os mecanismos internos de gerenciamento de memória utilizados em sistemas operacionais modernos.